

Coursework Feedback & Grade Report

Academic Year 2025/2026

Student Name:

Mikey Ibiayo

Module Leader:

Hamed Hamzeh

Assessment Summary					
Part 1:	10.0/10	Part 2:	17.5/20	Part 3:	19.0/20
Part 4:	20.0/20	Part 5:	26.5/30	OVERALL:	93/100

Detailed Feedback Breakdown

Section and criterion names below follow the official coursework rubric exactly.

PART 1: SETUP & DISCOVERY (10 Marks)					
Criteria	Max	Code	Report	Video	Feedback
1.1 Architecture & Config <i>Video Demo Requirements: N/A</i>	5	2.5	1.0	1.5	Code: Application configuration is excellent, correctly utilising the JAX-RS <code>@ApplicationPath</code> to standardise the entry point. Report: The report documentation provides a satisfactory overview of the architecture. Video: The video demonstration successfully highlights the API entry endpoint and JSON output.
1.2 Discovery Endpoint <i>Video Demo Requirements: N/A</i>	5	2.5	1.0	1.5	Code: The endpoint is implemented meticulously, returning proper <code>hateoas</code> links via the <code>DiscoveryResource</code> . Report: Responses in the README are adequately detailed. Video: Video demonstrates the discovery mechanism accurately.
Part 1 Section Total:					10.0 / 10

PART 2: ROOM MANAGEMENT (20 Marks)					
Criteria	Max	Code	Report	Video	Feedback
2.1 Room Implementation <i>Video Demo Requirements: POST a room: Show 201 Created & Location header. GET by new ID.</i>	10	4.0	2.0	3.0	Code: The <code>@POST</code> annotation correctly returns a 201 status with a <code>Location</code> header. However, it is a best practice to have a mechanism to generate id automatically. Report: The rationale for the implementation is well-articulated in the report. Video: Both creation and retrieval of the room are clearly shown.
2.2 Deletion & Logic <i>Video Demo Requirements: DELETE a room: Show success, then show 409 Conflict for rooms with sensors.</i>	10	5.0	2.0	1.5	Code: Deletion logic is robust; it successfully checks for dependent sensors and safely throws a <code>RoomNotEmptyException</code> . Report: The deletion workflow is clearly explained in the documentation. Video: The conflict scenario was demonstrated correctly, but unfortunately, a successful room deletion was omitted.
Part 2 Section Total:					17.5 / 20

PART 3: SENSORS & FILTERING (20 Marks)					
Criteria	Max	Code	Report	Video	Feedback
3.1 Sensor Integrity <i>Video Demo Requirements: POST sensor: Show error for non-existent roomId vs success for valid ID.</i>	10	4.0	2.0	3.0	Code: Thorough validation ensures a sensor cannot be attached to a non-existent room. However, it is a best practice to have a mechanism to generate id automatically. Report: Sensor integrity rules are analysed well in the report. Video: The video perfectly captures the success and dependency failure behaviours.
3.2 Filtered Retrieval <i>Video Demo Requirements: GET /sensors?type=...: Change parameter and show dynamic list update.</i>	10	5.0	2.0	3.0	Code: Outstanding use of <code>@QueryParam</code> and Java Streams to dynamically filter the sensor list. Report: The report provides strong justifications for the filtering approach. Video: Dynamic list updates based on the query parameter are clearly exhibited.
Part 3 Section Total:					19.0 / 20

PART 4: SUB-RESOURCES (20 Marks)					
Criteria	Max	Code	Report	Video	Feedback
4.1 Sub-Resource Locator <i>Video Demo Requirements: Navigate /sensors/{id}/readings. Show nested structure in Postman.</i>	10	5.0	2.0	3.0	Code: The sub-resource locator pattern is applied flawlessly, cleanly routing to <code>SensorReadingResource</code> . Report: The structural choices are thoroughly justified in the report. Video: The nested endpoint routing is correctly demonstrated.
4.2 Historical Management <i>Video Demo Requirements: Navigate /sensors/{id}/readings. Show nested structure in Postman.</i>	10	6.0	N/A	4.0	Code: Readings are meticulously managed and filtered, adhering to the expected nested structure. Report: N/A Video: Historical data retrieval is executed and visualised properly.
Part 4 Section Total:					20.0 / 20

PART 5: ERROR HANDLING (30 Marks)					
Criteria	Max	Code	Report	Video	Feedback
5.1 Resource Conflict (409) <i>Video Demo Requirements: Attempt to delete a Room with active Sensors assigned. Show 409 Conflict with JSON body.</i>	5	3.0	N/A	2.0	Code: Exception mapping for 409 Conflict is handled excellently, returning structured JSON. Report: N/A Video: The conflict error response is effectively proven.
5.2 Dependency Validation (422) <i>Video Demo Requirements: Client attempts to POST a new Sensor with a non-existent roomId. Show HTTP 422 or 400 JSON error.</i>	10	5.0	2.0	3.0	Code: <code>LinkedResourceNotFoundException</code> maps efficiently to a 422 status with comprehensive JSON feedback. Report: The report provides a deep analysis of dependency validation. Video: The 422 error response is captured and explained well.
5.3 State Constraint (403) <i>Video Demo Requirements: Attempt to POST a reading to a sensor marked as "MAINTENANCE". Show HTTP 403 Forbidden.</i>	5	3.0	N/A	0.0	Code: State validation is programmed well, restricting POST requests to active sensors. Report: N/A Video: This critical behaviour was entirely missing from the video demonstration.
5.4 The Global Safety Net (500) <i>Video Demo Requirements: Trigger a runtime error (e.g., NullPointerException) and show generic HTTP 500 without a raw stack trace.</i>	5	2.5	1.0	1.5	Code: <code>GlobalExceptionHandler</code> safely intercepts unexpected faults, preventing stack trace leakage. Report: Safety net logic is concisely documented. Video: The generic 500 error behaviour is successfully triggered and shown.
5.5 API Logging Filters <i>Video Demo Requirements: Show console logs demonstrating recorded HTTP method, URI, and final HTTP status code.</i>	5	2.5	1.0	0.0	Code: The <code>LoggingFilter</code> successfully implements container request and response interfaces. Report: Filter application is accurately detailed in the report. Video: Server logs and HTTP details were unfortunately not displayed in the demonstration.
Part 5 Section Total:					26.5 / 30

General Comments & Feedforward

An exceptional submission demonstrating a strong understanding of JAX-RS and client-server architectures. The codebase is highly cohesive, well-structured, and strictly adheres to the zero-tolerance constraints, correctly bypassing frameworks like Spring Boot in favour of core JAX-RS capabilities. Error handling is rigorous, with custom exception mappers returning precisely formatted JSON structures, and sub-resource routing is perfectly executed. Moving forward, it is a best practice to have a mechanism to generate id automatically for primary entities (such as Rooms and Sensors) to enhance data integrity and standardise API behaviour. Additionally, ensure video demonstrations meticulously cover all scenarios, particularly standard success flows (like resource deletion) and server logging visibility, to secure maximum marks. Overall, an excellent effort yielding a first-class result.